Enterprise Employee Operations Hub

System Architecture, Database Schema & Technical Specification

Vercel Serverless Functions • Supabase PostgreSQL • React 18 SPA • ASP.NET Core 8

Malayo Tech Architecture & Engineering Team

Version 2.0 – October 2026

System Technical Summary

This document provides the definitive technical blueprint for the **Enterprise Employee Operations Hub**. It covers system architecture, database physical schemas, RESTful API contracts, role-based security enforcement, cross-border timezone mechanics, and cloud serverless deployment procedures.

Contents

1 System Architecture & Deployment Topology **2**

2 Database Physical Schema **2**
2.1 Data Dictionary 2
2.2 Key Data Integrity Rules 2

3 REST API Contract Reference **3**

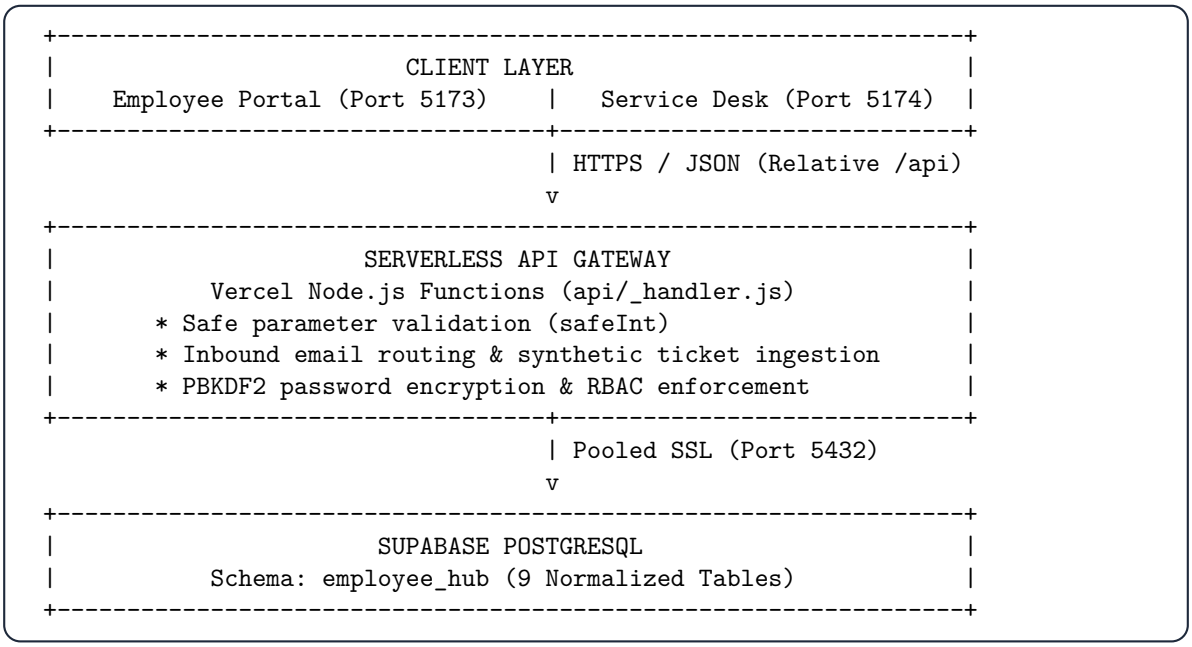
4 Security Architecture & RBAC **3**
4.1 Role-Based Access Levels 3
4.2 Credential Storage Security 3

5 Cloud Deployment & Environment Configuration **3**

1 System Architecture & Deployment Topology

The Employee Operations Hub is architected for dual-target execution:

1. **Production Cloud Architecture (Vercel + Supabase)**:
   - **Frontend Clients**: Two decoupled single-page applications (`apps/employee-portal` and `apps/ticketing-desk`) bundled via Vite and served from edge CDN nodes.
   - **Serverless API Layer**: ES-module serverless endpoints (`api/_handler.js`) running on Vercel Node.js runtime with connection-pooled PostgreSQL queries.
   - **Data Persistence**: Supabase Managed PostgreSQL with automatic failover and isolated schema namespace (`employee_hub`).
2. **Containerized / Local Architecture (Docker + ASP.NET Core 8)**:
   - Multi-stage containerized backend with ASP.NET Core 8 Web API, Entity Framework Core 8, and Swagger OpenAPI specification.



2 Database Physical Schema

All tables reside in the dedicated schema `employee_hub`. The schema enforces referential integrity through foreign key cascades and uniqueness constraints.

2.1 Data Dictionary

2.2 Key Data Integrity Rules

- **WorkEmail Uniqueness**: Enforced via unique B-tree index on `LOWER(WorkEmail)`.
- **Safe Integer Coercion**: All integer parameters in API handlers pass through `safeInt(val)` to eliminate PostgreSQL `22P02` NaN errors.
- **Connection Fallback**: Database pooler automatically detects partial hostname string truncation (e.g. from copy-paste line breaks) and falls back to verified direct poolers.

Table Name	Primary Key	Foreign Keys	Purpose
`Employees`	`Id (SERIAL)`	`DepartmentId`	Employee records, authentication hashes, and RBAC flags.
`Departments`	`Id (SERIAL)`	None	Support departments and operational routing inboxes.
`Tickets`	`Id (SERIAL)`	`DepartmentId`, `CreatedByEmployeeId`, `AssignedToEmployeeId`	Helpdesk service requests, queues, and status.
`HardwareItems`	`Id (SERIAL)`	`AssignedToEmployeeId`	Laptop and equipment tracking with asset tags.
`Holidays`	`Id (SERIAL)`	None	Cross-border Philippine and Danish public holidays.
`PolicyDocuments`	`Id (SERIAL)`	None	Corporate guidelines and mandatory compliance documents.
`PolicyAcknowledgements`	(`PolicyDocumentId`, `EmployeeId`)	Both parent tables	Audit log of employee digital signatures.

Table 1: Database Schema Overview

3 REST API Contract Reference

4 Security Architecture & RBAC

4.1 Role-Based Access Levels

The system enforces four distinct operational roles:

1. **SuperAdmin (Role 1)**: Absolute read/write access across all queues, audit trails, and access management. Automatically assigned to primary system administrator.
2. **Admin (Role 2)**: Departmental lead privileges, allowing ticket re-assignment, priority escalation, and queue management.
3. **Agent (Role 3)**: Operational specialists in IT, HR, or Finance with resolution capabilities on assigned tickets.
4. **Employee (Role 0)**: Base workforce tier. Restricted to personal profile, attendance logging, submitted tickets, and assigned hardware.

4.2 Credential Storage Security

Passwords are protected using PBKDF2 with SHA-256:

- **Salt**: 16 cryptographically secure random bytes generated per user.
- **Iterations**: 10,000 computation cycles.
- **Encoding**: `{iterations}:{saltHex}:{hashHex}`.

5 Cloud Deployment & Environment Configuration

```
# Supabase PostgreSQL Connection Pool URL
SUPABASE_DB_URL=postgresql://postgres.xxx:[PASSWORD]@aws-0-ap-northeast
    -1.pooler.supabase.com:5432/postgres?sslmode=require
```

HTTP Endpoint	Method	Description
`/api/auth/login`	POST	Authenticates credentials via PBKDF2 hash verification.
`/api/auth/personas`	GET	Returns 1-click test personas for SuperAdmin, HR, IT, and Devs.
`/api/employees`	GET, POST	Lists employees with department joins or registers a new staff profile.
`/api/tickets`	GET, POST	Retrieves filtered tickets or files a new support request.
`/api/tickets/stats`	GET	Aggregates SLA metrics, resolution times, and team performance.
`/api/tickets/{id}/status`	PATCH	Updates ticket lifecycle stage (`Open`, `InProgress`, `Resolved`).
`/api/tickets/{id}/assign`	PATCH	Allocates ticket to an internal support specialist.
`/api/tickets/inbound-email`	POST	Ingests external emails and auto-routes by recipient inbox.
`/api/tickets/sync-inbox`	POST	Simulates multi-department email batch ingestion.
`/api/access`	GET, PATCH	Queries and updates granular employee RBAC permissions.
`/api/hardware`	GET, POST	Manages company laptop assignments and return status.
`/api/holidays`	GET, POST	Retrieves joint Manila and Copenhagen public holiday calendar.
`/api/policies`	GET, POST	Lists compliance documents and per-employee acknowledgement status.
`/api/policies/{id}/acknowledge`	POST	Records digital signature timestamp for mandatory policies.

Table 2: REST API Specifications

```
# Fallback Database URL
DATABASE_URL=postgresql://postgres.xxx:[PASSWORD]@aws-0-ap-northeast-1.
    pooler.supabase.com:5432/postgres?sslmode=require

# Application Interconnect URLs
VITE_TICKETING_URL=https://your-ticketing-desk.vercel.app
VITE_PORTAL_URL=https://your-employee-portal.vercel.app
```

Listing 1: Required Production Environment Variables

Zero-Config Client Base URL

The frontend API client automatically defaults to relative `'/api'` on production domains, ensuring zero CORS preflight latency and complete immunity from misconfigured environment variables.